Procesamiento distribuido con Spark

Rodrigo Blanco Betes. Alfa Formación

Enero del 2026

1. Uso de Spark como framework de procesamiento de la información en sistemas distribuidos
2. Inmersión en la estructura de datos principal de Spark: RDD – Resilient Distributed Dataset
3. Iniciación a Spark en cluster
4. Conceptos de programación paralela en Spark
5. Optimización de rendimiento en Spark
6. Iniciación al procesamiento de datos en Spark mediante el uso del módulo SparkSQL

Principales retos en los sistemas distribuidos

- **Latencia y red**: comunicación entre nodos más lenta que la memoria local; penaliza algoritmos con muchas rondas.
- **Fallos parciales**: nodos que caen, discos que fallan, timeouts; el sistema debe *recuperar* sin parar todo.
- **Consistencia y coordinación**: sincronización, locks, consenso; evitar cuellos de botella por coordinación excesiva.
- **Distribución de datos**: particionado, *skew* (particiones desbalanceadas), localización (data locality).
- **Reproducibilidad y determinismo**: reintentos pueden duplicar efectos si no hay cuidado (*exactly-once* vs *at-least-once*).
- **Gestión de recursos**: memoria, CPU, disco, I/O; contención cuando varios jobs compiten.

Retos típicos que aparecen en la práctica

- **Shuffles caros**: joins/aggregations provocan intercambio masivo de datos por red.
- **Stragglers**: unas pocas tareas lentas alargan toda la etapa (*stage*).
- **Observabilidad**: entender "qué pasa" requiere métricas, UI, logs, y explicar planes de ejecución.
- **Coste**: más nodos no siempre implica más rendimiento (ley de rendimientos decrecientes).

¿Por qué Spark? Ventajas clave

- **Procesamiento en memoria**: acelera workloads iterativos (ML, graph, ETL encadenado).
- **Modelo unificado**: batch, streaming, SQL, ML (ecosistema integrado).
- **Tolerancia a fallos**: recomputación a partir de *lineage* (DAG) de transformaciones.
- **Optimización automática**: Catalyst (planes lógicos/físicos) y Tungsten (ejecución eficiente).
- **APIs de alto nivel**: DataFrames/Datasets (tipado en Scala), SQL, funciones built-in.
- **Escalabilidad**: desde local a clúster (YARN, Kubernetes, Standalone) sin reescribir el código.

Conceptos Spark que conectan con "beneficios"

- **DAG de ejecución**: el job se divide en **stages** (por barreras de shuffle) y **tasks**.
- **Transformations vs Actions**:
  - *Transformations* son **perezosas** (lazy): construyen el plan.
  - *Actions* disparan la ejecución: materializan el resultado.
- **Persistencia/caching**: control explícito de reutilización para evitar recomputaciones.
- **Particionado**: clave para rendimiento (paralelismo) y para evitar *skew*.

Ejemplo mínimo: lazy + action (Scala)

```
val df = spark.read.parquet("/data/events")

val cleaned = df
  .filter($"country".isNotNull)
  .withColumn("day", to_date($"ts"))

val agg = cleaned.groupBy($"day").count()

agg.show()   // Action: aquí se ejecuta el plan
```

Cuándo Spark brilla (y cuándo no)

- **Brilla**: ETL grande, joins/aggregations a escala, pipelines, análisis exploratorio, streaming.
- **No siempre ideal**: latencia ultra-baja (ms), OLTP, tareas pequeñas donde el overhead domina.
- **Regla práctica**: Spark es una "fábrica" para datos; no un motor transaccional.

Fundamentos de programación funcional (FP)

- **Funciones como valores**: se pasan como parámetros y se retornan.
- **Inmutabilidad**: evita efectos colaterales y simplifica concurrencia.
- **Funciones puras**: mismo input $\Rightarrow$ mismo output; facilita pruebas y reintentos.
- **Transformaciones sobre colecciones**: `map`, `filter`, `flatMap`, `reduce`.
- **Composición**: construir pipelines encadenando transformaciones.

FP y sistemas distribuidos: por qué encaja tan bien

- **Menos estado compartido** ⇒ menos coordinación entre nodos.
- **Reintentos seguros** si las funciones son puras (muy alineado con tolerancia a fallos).
- **Pipelines declarativos**: describes *qué* hacer; el motor decide *cómo* ejecutarlo.
- **Paralelismo natural**: operaciones por elemento/partición se distribuyen fácilmente.

Ejemplo FP: de listas a RDD/DataFrame (Scala)

```
// Colección local (FP clásica)
val xs = List(1,2,3,4,5)
val ys = xs.filter(_ % 2 == 0).map(_ * 10)

// Idea similar en Spark (distribuido)
val rdd = sc.parallelize(xs)
val out = rdd.filter(_ % 2 == 0).map(_ * 10)
```

Idea clave para Spark: "declarar transformaciones"

- En Spark, muchas operaciones son **transformaciones** (lazy) sobre datos distribuidos.
- La FP ayuda a pensar en términos de:
  - **pipeline** de pasos
  - **datos inmutables**
  - **funciones** que transforman datos
- Resultado: código más **razonable**, **testeable** y **escalable**.

¿Qué es un RDD?

- **RDD (Resilient Distributed Dataset)**: colección **distribuida** e **inmutable** de elementos.
- **Particionado**: los datos se dividen en particiones procesadas en paralelo.
- **Resilient**: tolerancia a fallos mediante **lineage** (DAG de transformaciones).
- Nivel de API más **bajo** que DataFrames/Datasets: más control, menos optimización automática.

Tipos de operaciones sobre RDDs

- **Transformations** (lazy): construyen el plan, no ejecutan.
  - `map`, `flatMap`, `filter`
  - `distinct`, `union`, `intersection`
  - `groupByKey`, `reduceByKey`, `join`
- **Actions**: disparan la ejecución y devuelven resultado o escriben salida.
  - `count`, `collect`, `take`, `first`
  - `reduce`, `saveAsTextFile`
- **Persistencia**: `cache`/`persist` para reutilizar resultados entre acciones.

Operaciones "narrow" vs "wide" (impacto en rendimiento)

- **Narrow transformations**: cada partición de salida depende de una sola partición de entrada.
  - Ejemplos: `map`, `filter`
  - Ventaja: se ejecutan **sin shuffle**, suelen ser más rápidas.
- **Wide transformations**: la salida depende de múltiples particiones de entrada.
  - Ejemplos: `reduceByKey`, `groupByKey`, `join`
  - Implican **shuffle**: intercambio de datos por red (coste alto).

Introducción a MapReduce

- **Map**: transformar cada elemento en pares clave–valor (o en una colección de pares).
- **Shuffle/Sort**: agrupar por clave (fase costosa).
- **Reduce**: combinar todos los valores asociados a cada clave (agregación).
- Patrón típico: **contar**, **sumar**, **agrupar**, **calcular métricas por categoría**.

MapReduce en Spark (RDDs)

- Spark implementa este patrón con transformaciones sobre RDDs:
  - `map/flatMap` para generar claves y valores.
  - `reduceByKey` o `aggregateByKey` para reducir por clave.
- Diferencia clave: Spark puede **mantener datos en memoria** y encadenar múltiples etapas de forma eficiente.
- **Buenas prácticas**:
  - Preferir `reduceByKey` frente a `groupByKey` (menos datos en shuffle).
  - Controlar el particionado (`repartition/coalesce`) si hay *skew*.

Ejemplo: WordCount (MapReduce clásico) con RDD (Scala)

```
val lines = sc.textFile("/data/book.txt")

val counts = lines
  .flatMap(_.split("\\s+"))
  .map(word => (word.toLowerCase, 1))
  .reduceByKey(_ + _)

counts.take(10).foreach(println)
```

Aplicación práctica: caso típico de logs

- Objetivo: **nº de eventos por tipo** y **top N** más frecuentes.
- Flujo típico:
  1. Leer logs (texto/JSON) $\rightarrow$ parseo.
  2. Mapear a $(tipoEvento, 1)$.
  3. `reduceByKey` para contar.
  4. Ordenar y seleccionar top N.
- Métricas a vigilar: particiones, shuffle, tiempo por stage, *skew*.

Ejemplo práctico: top 5 eventos (Scala)

```
val rdd = sc.textFile("/data/events.log")

val top5 = rdd
  .map(parseEventType)          // String => eventType
  .map(t => (t, 1))
  .reduceByKey(_ + _)
  .map { case (t, c) => (c, t) }
  .sortByKey(ascending = false)
  .take(5)

top5.foreach(println)
```

Arquitectura básica de Spark en cluster

- **Driver**: proceso principal que contiene el `SparkSession` y construye el DAG.
- **Cluster Manager**: gestiona recursos (CPU, memoria) en los nodos.
- **Executors**: procesos en los nodos worker que ejecutan tareas.
- **Tasks**: unidad mínima de ejecución, una por partición.
- Flujo general:
  1. El driver solicita recursos al cluster manager.
  2. Se lanzan executors.
  3. El driver envía tareas a los executors.
  4. Los executors procesan particiones y devuelven resultados.

Gestores de recursos soportados

- **Spark Standalone**
  - Cluster manager propio de Spark.
  - Sencillo de configurar.
  - Adecuado para entornos dedicados.
- **Hadoop YARN**
  - Integrado en ecosistemas Hadoop.
  - Comparte recursos con otras aplicaciones.
  - Muy habitual en entornos empresariales.
- **Apache Mesos**
  - Cluster manager generalista.
  - Permite compartir recursos entre múltiples frameworks.

Funcionamiento del Driver de Spark

- Punto de entrada de la aplicación.
- Construye el **DAG** de transformaciones.
- Divide el job en **stages** separados por shuffles.
- Planifica tareas y las asigna a executors.
- Mantiene metadatos, estado de ejecución y resultados parciales.
- Si el driver falla, la aplicación termina (salvo configuraciones específicas en cluster).

Modos de despliegue del Driver

- **Client mode**:
  - El driver se ejecuta fuera del cluster (por ejemplo, en la máquina local).
  - Útil para desarrollo y debugging.
- **Cluster mode**:
  - El driver se ejecuta dentro del cluster.
  - Más robusto para producción.

Principales propiedades de configuración

- `spark.master`: URL del cluster manager.
- `spark.app.name`: nombre de la aplicación.
- `spark.executor.memory`: memoria por executor.
- `spark.executor.cores`: núcleos por executor.
- `spark.driver.memory`: memoria del driver.
- `spark.executor.instances`: número de executors.
- `spark.default.parallelism`: paralelismo por defecto.

Ejemplo de envío de aplicación al cluster

```
spark-submit \
  --class com.ejemplo.Main \
  --master yarn \
  --deploy-mode cluster \
  --executor-memory 4G \
  --executor-cores 2 \
  --num-executors 4 \
  app.jar
```

Compilación y empaquetamiento

- Aplicaciones Scala/Java:
  - Compilación con Maven o SBT.
  - Generación de JAR con dependencias (fat jar o shaded jar).
- Dependencias externas:
  - Marcar Spark como `provided` si el cluster ya lo incluye.
- Python:
  - Scripts enviados con `spark-submit`.
  - Posible uso de entornos virtuales o `-py-files`.

Logging en aplicaciones Spark

- Spark utiliza **log4j** / **log4j2** para el sistema de logs.
- Permite configurar:
  - Nivel de log (INFO, WARN, ERROR, DEBUG).
  - Appenders (consola, fichero).
- Importante para:
  - Diagnóstico de errores.
  - Análisis de rendimiento.
  - Seguimiento en producción.
- Complemento visual: Spark UI (puerto 4040 en local).

Resumen conceptual

- El driver orquesta; los executors ejecutan.
- El cluster manager asigna recursos.
- La configuración adecuada impacta directamente en rendimiento y estabilidad.
- El empaquetado y logging correctos son esenciales para producción.

Paralelismo en Spark: idea fundamental

- Spark paraleliza el procesamiento a nivel de **particiones**.
- Cada partición se procesa mediante una **task**.
- El número de particiones influye directamente en:
  - Nivel de paralelismo.
  - Uso de CPU.
  - Overhead de planificación.
- Regla práctica: más particiones $\neq$ siempre mejor rendimiento.

Particionado de los RDDs cargados desde fichero

- Cuando se usa `sc.textFile()` o similares:
  - Spark crea una partición por **bloque de fichero** (por defecto).
- Puede indicarse número mínimo de particiones:
  - `sc.textFile(path, minPartitions)`
- Influye:
  - Tamaño del fichero.
  - Tamaño del bloque en el sistema de almacenamiento.
- Buen diseño de particionado mejora el paralelismo inicial.

Ejemplo: controlando particiones

```
val rdd = sc.textFile("/data/logs.txt", 8)

println(rdd.getNumPartitions)

// Reparticionar después
val repartitioned = rdd.repartition(16)
```

Particionado de la información leída desde HDFS

- HDFS divide los ficheros en **bloques** (por ejemplo, 128MB).
- Cada bloque suele corresponder a una partición inicial en Spark.
- Ventaja clave: **data locality**.
  - Las tareas se intentan ejecutar en el nodo que contiene el bloque.
  - Reduce tráfico de red.
- El paralelismo inicial depende del número de bloques.

Repartition vs Coalesce

- **repartition(n)**:
  - Aumenta o disminuye particiones.
  - Siempre implica shuffle.
- **coalesce(n)**:
  - Reduce particiones.
  - Puede evitar shuffle si se usa correctamente.
- Uso típico:
  - Reducir particiones antes de escribir a disco.

Cálculo de los Stages en Spark

- Spark construye un **DAG** de transformaciones.
- Divide el job en **stages** separados por operaciones que requieren shuffle.
- Regla general:
  - Transformaciones **narrow** $\rightarrow$ mismo stage.
  - Transformaciones **wide** (shuffle) $\rightarrow$ nuevo stage.
- Cada stage se compone de múltiples **tasks**.

Ejemplo conceptual de división en stages

- Código:
  1. `map`
  2. `filter`
  3. `reduceByKey`
  4. `map`
- Resultado:
  - Stage 1: `map` + `filter`
  - Stage 2: `reduceByKey` (shuffle)
  - Stage 3: `map` posterior

Relación entre particiones y stages

- Cada stage genera tantas tasks como particiones tenga el RDD de entrada.
- Más particiones $\rightarrow$ más tasks $\rightarrow$ mayor paralelismo.
- Pero también:
  - Más overhead.
  - Más coste de coordinación.
- El equilibrio es clave en programación paralela.

Ideas clave para programación paralela eficiente

- Diseñar bien el particionado inicial.
- Minimizar shuffles innecesarios.
- Evitar *data skew*.
- Ajustar número de particiones según recursos del cluster.
- Usar Spark UI para analizar stages y tareas.

El linaje (Lineage) en Spark

- Cada RDD mantiene un **linaje de transformaciones** que describe cómo fue construido.
- Este linaje forma un **DAG** (Directed Acyclic Graph).
- Permite:
  - **Tolerancia a fallos**: recomputar particiones perdidas.
  - Optimización de ejecución.
- Implicación práctica:
  - Si no se persiste un RDD, puede recomputarse varias veces.

Coste del linaje largo

- Pipelines muy largos $\rightarrow$ DAG complejo.
- Si hay múltiples acciones sobre el mismo RDD:
  - Se recalcula desde el origen si no está persistido.
- Solución:
  - **cache()** o **persist()**.
  - **checkpoint()** para cortar el linaje en procesos muy largos.

Estrategias de almacenamiento de RDDs

- Spark permite distintos niveles de persistencia:
  - `MEMORY_ONLY`
  - `MEMORY_AND_DISK`
  - `DISK_ONLY`
  - Versiones serializadas
- Elección depende de:
  - Tamaño del dataset.
  - Memoria disponible.
  - Coste de recomputación.

Ejemplo de persistencia

```
val rdd = sc.textFile("/data/logs.txt")
  .map(parse)
  .filter(valid)

rdd.cache()

val count1 = rdd.count()
val count2 = rdd.filter(_.level == "ERROR").count()
```

Buenas prácticas de persistencia

- Persistir solo si:
  - Se reutiliza varias veces.
  - La recomputación es costosa.
- Liberar memoria cuando ya no se necesita:
  - `rdd.unpersist()`
- Evitar cachear datasets enormes sin memoria suficiente.

Variables de Broadcast

- Permiten enviar una copia **eficiente y distribuida** de un objeto a todos los executors.
- Evitan:
  - Enviar el mismo objeto en cada task.
  - Sobrecarga de red innecesaria.
- Uso típico:
  - Tablas pequeñas de referencia.
  - Diccionarios de lookup.

Ejemplo de Broadcast

```
val lookup = Map("ES" -> "Spain", "FR" -> "France")

val broadcastVar = sc.broadcast(lookup)

val enriched = rdd.map { row =>
  val country = broadcastVar.value.get(row.code)
  (row, country)
}
```

Acumuladores (Accumulators)

- Variables que permiten agregar información desde los executors al driver.
- Uso típico:
  - Contadores.
  - Métricas de calidad.
- Características:
  - Solo el driver puede leer el valor final.
  - Los executors solo pueden incrementar.

Ejemplo de Acumulador

```
val errorCounter = sc.longAccumulator("ErrorCounter")

rdd.foreach { row =>
  if (!isValid(row)) {
    errorCounter.add(1)
  }
}

println(errorCounter.value)
```

Mecanismos simples para mejorar rendimiento

- Minimizar shuffles.
- Ajustar número de particiones.
- Persistir correctamente.
- Usar broadcast en joins pequeños.
- Utilizar acumuladores para métricas sin romper paralelismo.
- Monitorizar con Spark UI.

¿Qué es SparkSQL?

- Módulo de Spark para procesamiento estructurado.
- Introduce el concepto de **DataFrame**.
- Basado en optimización automática mediante:
  - **Catalyst Optimizer**
  - **Tungsten Execution Engine**
- Permite trabajar con:
  - API en Scala/Python
  - SQL directamente

Construcción de DataFrames en Spark

- Desde ficheros:
  - `read.csv()`
  - `read.json()`
  - `read.parquet()`
- Desde RDDs.
- Desde colecciones locales.
- Desde tablas externas (Hive, JDBC).

Ejemplo: Crear DataFrame desde fichero

```
val df = spark.read
  .option("header", "true")
  .option("inferSchema", "true")
  .csv("/data/users.csv")

df.printSchema()
```

Transformaciones comunes sobre DataFrames

- **select**: seleccionar columnas.
- **selectExpr**: expresiones SQL.
- **withColumn**: añadir/modificar columnas.
- **where / filter**: filtrar filas.
- **join**: combinar DataFrames.
- **unionAll**: unir datasets verticalmente.
- **groupBy**: agregaciones.
- **orderBy**: ordenar resultados.

Ejemplo de transformaciones

```
import org.apache.spark.sql.functions._

val transformed = df
  .select("name", "age", "country")
  .withColumn("age_plus_1", col("age") + 1)
  .filter(col("age") > 18)
  .groupBy("country")
  .count()
  .orderBy(desc("count"))
```

Tipos de joins en SparkSQL

- **inner join**
- **left / right outer join**
- **full outer join**
- **cross join**
- **broadcast join** (optimización automática o explícita)

Acciones comunes sobre DataFrames

- **show**: visualizar resultados.
- **collect**: traer datos al driver.
- **count**: contar filas.
- **first** / **head**: primera fila.
- **take**: primeras N filas.
- **write**: guardar en almacenamiento.

Ejemplo de acciones

```
transformed.show()

val total = transformed.count()

val firstRow = transformed.first()

transformed.write
  .mode("overwrite")
  .parquet("/data/output")
```

Buenas prácticas en SparkSQL

- Preferir DataFrames frente a RDDs en procesamiento estructurado.
- Evitar `collect()` en datasets grandes.
- Aprovechar funciones built-in antes que UDFs.
- Revisar el plan con:
  - `df.explain(true)`
- Monitorizar ejecución en Spark UI.